HDOC DAY-72

DYNAMIC STUDENT ATTENDANCE MANAGEMENT USING POINTERS IN C

Programming in C - Practical Activity

Name	Yashashwee Sharma
SAP ID	590032011
Batch	16

Submitted as HDOC Activity - Day 72

1. Aim

To develop a C program for managing student attendance using a dynamically resizable matrix. The program uses pointers, malloc(), realloc(), and free() so that the attendance records can grow when new students are added.

2. Problem Statement

Each row represents one student and each column represents one working day. The value 1 represents Present and 0 represents Absent. The program must create the initial attendance matrix dynamically, record valid attendance, add new students using realloc(), display the records, calculate each student's attendance percentage, and release all allocated memory.

3. Concepts Used

- Dynamic memory allocation using malloc() and realloc()
- Pointers and pointer arithmetic for matrix handling
- Functions and function prototypes
- Input validation using conditional statements and loops
- Attendance percentage calculation
- Memory deallocation using free()

4. Algorithm

1. Start the program.

2. Read the initial number of students and working days. Reject non-positive values.

3. Allocate memory for students x days integers using malloc().

4. Record attendance for each student and day. Accept only 0 or 1.

5. Display the initial attendance matrix.

6. Read the number of additional students.

7. If new students are to be added, increase the allocated block using realloc().

8. Record attendance for the newly added students.

9. Display the final attendance matrix.

10. Count the number of present days for each student and calculate: (Present Days / Working Days) x 100.

11. Free the dynamically allocated memory and terminate the program.

5. C Program

```
#include <stdio.h>
#include <stdlib.h>
```

```c
int *createMatrix(int students, int days);
void recordAttendance(int *attendance, int start, int students, int days);
int *addStudents(int *attendance, int oldStudents, int newStudents, int days);
void displayAttendance(int *attendance, int students, int days);
void calculatePercentage(int *attendance, int students, int days);
void releaseMemory(int *attendance);

int main()
{
    int students, days, additionalStudents;
    int *attendance;

    printf("Enter the initial number of students: ");
    if (scanf("%d", &students) != 1 || students <= 0)
    {
        printf("Invalid number of students.\n");
        return 1;
    }

    printf("Enter the number of working days: ");
    if (scanf("%d", &days) != 1 || days <= 0)
    {
        printf("Invalid number of working days.\n");
        return 1;
    }

    attendance = createMatrix(students, days);
    if (attendance == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("\nEnter attendance for initial students:\n");
    recordAttendance(attendance, 0, students, days);

    printf("\nInitial Attendance Record:\n");
    displayAttendance(attendance, students, days);

    printf("\nEnter number of additional students: ");
    if (scanf("%d", &additionalStudents) != 1 || additionalStudents < 0)
    {
        printf("Invalid input.\n");
        releaseMemory(attendance);
        return 1;
    }

    if (additionalStudents > 0)
    {
        int *temp = addStudents(attendance, students, additionalStudents, days);
        if (temp == NULL)
        {
            printf("Memory reallocation failed.\n");
            releaseMemory(attendance);
            return 1;
        }
        attendance = temp;

        printf("\nEnter attendance for new students:\n");
        recordAttendance(attendance, students, additionalStudents, days);
        students = students + additionalStudents;
    }

    printf("\nFinal Attendance Record:\n");
    displayAttendance(attendance, students, days);

    printf("\nAttendance Percentages:\n");
    calculatePercentage(attendance, students, days);

    releaseMemory(attendance);
```

```
    return 0;
}

int *createMatrix(int students, int days)
{
    return (int *)malloc(students * days * sizeof(int));
}

void recordAttendance(int *attendance, int start, int students, int days)
{
    int i, j, value;
    for (i = start; i < start + students; i++)
    {
        printf("\nStudent %d:\n", i + 1);
        for (j = 0; j < days; j++)
        {
            while (1)
            {
                printf("Day %d (1 = Present, 0 = Absent): ", j + 1);
                if (scanf("%d", &value) == 1 && (value == 0 || value == 1))
                {
                    *(attendance + i * days + j) = value;
                    break;
                }
                else
                {
                    printf("Invalid input! Enter only 0 or 1.\n");
                    while (getchar() != '\n');
                }
            }
        }
    }
}

int *addStudents(int *attendance, int oldStudents,
                 int newStudents, int days)
{
    int totalStudents = oldStudents + newStudents;
    return (int *)realloc(attendance, totalStudents * days * sizeof(int));
}

void displayAttendance(int *attendance, int students, int days)
{
    int i, j;
    printf("\nStudent\t");
    for (j = 0; j < days; j++)
        printf("Day %d\t", j + 1);
    printf("\n");

    for (i = 0; i < students; i++)
    {
        printf("%d\t", i + 1);
        for (j = 0; j < days; j++)
            printf("%d\t", *(attendance + i * days + j));
        printf("\n");
    }
}

void calculatePercentage(int *attendance, int students, int days)
{
    int i, j, present;
    float percentage;
    for (i = 0; i < students; i++)
    {
        present = 0;
        for (j = 0; j < days; j++)
        {
            if (*(attendance + i * days + j) == 1)
                present++;
        }
        percentage = ((float)present / days) * 100;
```

```
        printf("Student-%d: %.1f%%\n", i + 1, percentage);
    }
}

void releaseMemory(int *attendance)
{
    free(attendance);
}
```

6. Sample Test Case

Initial students = 3, Working days = 3, Additional students = 3

Student	Day 1	Day 2	Day 3
Student 1	1	0	0
Student 2	1	0	1
Student 3	1	1	1
Student 4	1	0	0
Student 5	1	1	1
Student 6	1	0	1

7. Expected Output

```
Final Attendance Record:
Student    Day 1    Day 2    Day 3
1          1        0        0
2          1        0        1
3          1        1        1
4          1        0        0
5          1        1        1
6          1        0        1

Attendance Percentages:
Student-1: 33.3%
Student-2: 66.7%
Student-3: 100.0%
Student-4: 33.3%
Student-5: 100.0%
Student-6: 66.7%
```

8. Function-wise Explanation

createMatrix() - Allocates the initial one-dimensional memory block for the attendance matrix using malloc().

recordAttendance() - Stores attendance using pointer arithmetic and repeatedly asks for input until 0 or 1 is entered.

addStudents() - Expands the existing memory block using realloc() when new students are added.

displayAttendance() - Displays the attendance matrix in student-by-day form.

calculatePercentage() - Counts present days and calculates the attendance percentage of every student.

releaseMemory() - Releases the dynamically allocated memory using free().

9. Key Pointer Expression

***(attendance + i * days + j)**

This expression accesses the attendance value at row i and column j without using a fixed-size 2D array. The matrix is stored as one continuous block of dynamically allocated memory.

10. Result / Conclusion

The program successfully manages student attendance using dynamic memory allocation and pointers. It creates the initial matrix using malloc(), increases its size using realloc() when students are added, validates attendance input, calculates attendance percentages, and finally releases the allocated memory using free().